

Eloquent Relationships

In this lesson, we will learn about Eloquent relationships which allow you to define relationships between models. This is a powerful feature of Eloquent that allows you to easily work with related data. In our app, we want to be able to associate a job listing with a user.

Types of Relationships

There are several types of relationships that you can define in Eloquent. These relationships do not apply only to Eloquent or Laravel or PHP. They are part of the database design and are used to model the relationships between tables in a database. The most common types of relationships are:

- **One-to-One:** A one-to-one relationship is a relationship between two tables where each record in one table is related to exactly one record in the other table. An example would be user profiles. A single user can have a single profile.
- **One-to-Many:** A one-to-many relationship is a relationship between two tables where each record in one table is related to zero, one, or many records in the other table. An example would be blog posts. A single user can write multiple blog posts, but each blog post is written by a single user.
- **Many-to-Many:** A many-to-many relationship is a relationship between two tables where each record in one table is related to zero, one, or many records in the other table, and vice versa. An example would be students and courses. A student can enroll in many courses, and each course can have many students.

Defining Relationships In Models

In our case, we want to define a one-to-many relationship between the **JobListing** model and the **User** model. This means that each **JobListing** can only have one **User** and each **User** can have many **JobListings**.

What this will allow us to do within our application is to easily access the user that created a job listing. We can also easily access all of the job listings that a user has created.

For instance:

```
$user = User::find(1);  
$jobListings = $user->jobListings;
```

or

```
$jobListing = JobListing::find(1);  
$user = $jobListing->user;
```

To define this relationship, we need to add a method to the **JobListing** model that defines the relationship. Add the following method to the **JobListing** model:

```
public function user(): BelongsTo
{
    return $this->belongsTo(User::class);
}
```

Import the `BelongsTo` class at the top of the file:

```
use Illuminate\Database\Eloquent\Relations\BelongsTo;
```

This method defines a `belongsTo` relationship between the `JobListing` model and the `User` model. The `belongsTo` method tells Eloquent that the `JobListing` model belongs to the `User` model. This means that each `JobListing` record will have a `user_id` column that references the `id` column of the `User` model.

Next, we need to define the inverse of this relationship in the `User` model. Add the following method to the `User` model:

```
public function jobListings(): HasMany
{
    return $this->hasMany(JobListing::class);
}
```

Import the `HasMany` class at the top of the file:

```
use Illuminate\Database\Eloquent\Relations\HasMany;
```

This method defines a `hasMany` relationship between the `User` model and the `JobListing` model. The `hasMany` method tells Eloquent that the `User` model has many `JobListing` records. This means that each `User` record can have many `JobListings` associated with it.

Update Migration

Now that we have defined the relationship between the `JobListing` and `User` models, we need to update the `job_listings` table migration to add a `user_id` column. Add the following code to the `up` method of the migration:

```
Schema::table('job_listings', function (Blueprint $table) {
    // Add this line
    $table->unsignedBigInteger('user_id')->after('id');
    //...
```

We are specifying a new field called `user_id` that is an unsigned big integer which is a positive number with a wide range. We are also specifying that it should be added after the `id` column.

Next, we need to add a foreign key constraint to the `user_id` column. Add the following code at the bottom of the `up` method:

```
Schema::table('job_listings', function (Blueprint $table) {  
    // ...  
  
    // Adding a foreign key constraint  
    $table->foreign('user_id')->references('id')->on('users')->onDelete('cascade');  
});
```

This code adds a foreign key constraint to the `user_id` column. The `references` method specifies that the `user_id` column references the `id` column of the `users` table. The `onDelete('cascade')` method specifies that if a `User` record is deleted, all of its associated `JobListing` records should also be deleted.

Finally, we need to update the `down` method of the migration to remove the `user_id` column and the foreign key constraint. Add the following code to the `down` method:

```
public function down(): void  
{  
    Schema::table('job_listings', function (Blueprint $table) {  
        // Drop foreign key constraint and user_id column  
        $table->dropForeign(['user_id']);  
        $table->dropColumn('user_id');  
  
        // ...  
    });  
}
```

One last thing you need to do is add the `user_id` to the `$fillable` property of the `JobListing` model. Add the following code to the `JobListing` model:

```
protected $fillable = [  
    'user_id',  
    // ...  
];
```

Now we are ready to run our migration. Run the following command:

```
php artisan migrate
```

Now you should have all of those fields in your `job_listing` table. Now that our table is all set, I want to start to seed some data using factories.